Sarabo — Presentation Script (exactly 5 minutes)

10 slides · about 700 spoken words · speak at about 140 words per minute.
The time at each heading is when that slide should **start**. If you are more than 10 seconds behind at slide 6, skip the second sentence marked *(cut if late)* on slides 6–9.

Slide 1 — Title · 0:00 (15 s)

Assalamu alaikum. I am Jahid Hasan, ID 222-115-116. My final-year project, supervised by Shrabanti Chowdhury, is **Sarabo — an electronics repair service management platform.**

Slide 2 — The problem · 0:15 (25 s)

Getting a device repaired today is a leap of faith. Customers hand over a phone without knowing the price, or where the device is. Technicians find work only by word of mouth, and payment is uncertain. And whoever runs the service cannot see stuck jobs, and does refunds by hand. The result is low trust on both sides, and many repairs never happen.

Slide 3 — The solution · 0:40 (25 s)

Sarabo turns this into **one tracked, priced workflow for three roles.** The customer requests, compares, pays and tracks. The technician finds jobs, quotes, repairs and gets paid. The admin approves technicians, sets prices, handles problems and reads reports. There are 45 services across 8 device types, priced in taka, and the system is live on Vercel.

Slide 4 — How it works · 1:05 (40 s)

Here is the full journey in six steps. **One:** the customer describes the problem — even in Banglish — and AI fills the form. **Two:** local technicians with the right skill send offers. **Three:** the customer pays a small inspection fee to book. **Four:** after inspection, the technician sends an itemised quote. **Five:** the customer pays only what is left, because the fee is already counted. **Six:** hand-over — the technician gets 90 percent and a 30-day warranty starts.
It is fair by design: a no-show or an early cancel means a full refund.

Slide 5 — ER diagram · 1:45 (35 s)

This is the data model in MongoDB. A **user** creates many repair requests and can also become a **technician**. Each **repair request** is priced by one **service**, receives many **offers**, and has its **payments**, at most one **review**, and at most two **warranty claims**. The quote and inspection are stored inside the request itself, because they belong only to it. The crow's foot marks the "many" side.

Slide 6 — Customer: ranked offers · 2:20 (40 s)

When offers arrive, Sarabo ranks them using five things it already stores: rating, reliability, price, district and reply time — and it shows the reasons as small tags. The customer still makes the choice.

(cut if late) After the inspection, the quote shows exactly what is left: here a 4,000-taka quote, minus the 200-taka fee already paid, leaves 3,800. Approve and pay happen in one step.

Slide 7 — Warranty and no stuck repairs · 3:00 (35 s)

Every repair has a **30-day warranty.** The same technician re-inspects for free, and a covered fault is repaired again or refunded.
(cut if late) I also made sure no request can wait forever. Every waiting state has a deadline: an unpaid quote closes after 3 days, receipt is confirmed automatically after 3 days, a technician who can't finish triggers a refund, and stalled work alerts the admin.

Slide 8 — Applied AI · 3:35 (30 s)

Google Gemini helps in four places: the request helper, the technician's offer note, the support chat and the admin's report summary. But the rule is simple — **AI suggests, people decide.**
(cut if late) AI never sets a price, every answer is checked against live data, no personal data is sent, and usage has daily limits.

Slide 9 — Architecture · 4:05 (30 s)

A React 19 client talks to an Express 5 API on Vercel in Mumbai, right next to the MongoDB Atlas database. Because this system handles money, the **server decides** — it checks role and state on every action. Booking, payment and refunds run in **database transactions**, so they are all-or-nothing.
(cut if late) And payments and bookings happen **exactly once**, even if two people click at the same moment.

Slide 10 — Results · 4:35 (25 s)

All 2,761 server tests and all 58 browser tests pass, all twelve objectives were met, and the system is live. The limits: Stripe is in test mode, bKash payouts are manual, and there are no real customers yet. Next, I would add a local payment gateway and a mobile app with a full Bangla interface.
Thank you. Now I will show you the live demo.

[5:00]

Rehearsal checklist

- Run it with a timer three times. Mark your time at slide 6 (target 2:20) and slide 10 (target 4:35).
- Pause for one breath after each slide title — it is already counted in the timing.
- Numbers to say clearly: **45** services, **8** device types, **90 percent**, **30-day**, **2,761** and **58** tests.
- Do not read the slide; look at the audience and point at the screenshot.